

Comparing r-Algorithm and su-Algorithm for Optimization Problems

Project Report

Xiaoyan Lin

Advisor: Anton Malandii

Stony Brook University

May 2025

1 Introduction

Optimization problems with smooth objective functions are solved using gradient-based methods, where derivatives are well-defined and efficient descent directions can be computed. Classical approaches such as Gradient Descent, Newton’s method, and Quasi-Newton (variable metric) methods have been widely used due to their strong convergence properties in smooth settings. However, when the objective function becomes nonsmooth, these methods are no longer directly applicable or may become unstable, since gradients may not exist or may be discontinuous. In such cases, subgradient methods provide a natural extension by replacing gradients with subgradients. Nevertheless, classical subgradient methods often suffer from slow convergence, particularly for ill-conditioned problems with ravine-like structures. In these scenarios, the iterates tend to exhibit zigzagging behavior, resulting in inefficient progress toward the optimum. Moreover, second-order methods such as Newton or quasi-Newton approaches can be computationally expensive, as they require matrix inversions or approximations of Hessians, and their performance may degrade significantly in nonsmooth settings. To overcome these difficulties, inspired by the steepest descent method and conjugate gradient methods for solving smooth quadratic problems, Shor proposed the r-algorithm [8]. This method combines subgradient descent with a transformation of the variable space. Instead of updating the solution directly in the original space, the algorithm performs descent in a transformed space defined by a matrix. This transformation is updated iteratively using the difference between successive subgradients, which helps reshape the geometry of the problem. As a result, the space dilation technique reduces the elongation of level sets and produces more effective descent directions, significantly improving convergence behavior compared to classical subgradient methods. Another important approach is the variable metric method proposed in *“New Variable-Metric Algorithms for Nondifferentiable Optimization Problems”* by Uryasev [12]. This method introduces a different strategy for updating the transformation matrix. Instead of applying space dilation based on the difference of subgradients, as in the r-algorithm, the transformation matrix is updated using subgradient information, where the update incorporates both the current and the previous gradients. We refer to this approach as the su-algorithm.

This project studies and compares these two algorithms (the r-algorithm and the su-algorithm) from both theoretical and experimental perspectives. By implementing both algorithms in Python and evaluating their performance on a variety of problems, including smooth ill-conditioned functions, nonsmooth ill-conditioned functions, machine learning, and optimization problems. Our goal is to understand how different mechanisms for space transformation in the r-algorithm and gradient-based matrix update in the su-algorithm affect convergence behavior. For selected cases, we also benchmark their performance against the solution obtained by using Portfolio Safeguard (PSG) [1]. Furthermore, we investigate the impact of limited-memory updates for the transformation matrix. In practice, storing and updating the full matrix can be computationally expensive, especially in high-dimensional settings. Therefore, we study how restricting the memory used in updating the

transformation matrix affects convergence behavior and final solution quality, and analyze the trade-off between computational efficiency and accuracy. The numerical experiments show that the r-algorithm often exhibits faster and more stable convergence than the su-algorithm, especially on the ill-conditioned smooth and nonsmooth test problems. On smooth and nonsmooth ill-conditioned functions, the r-algorithm reaches near-optimal accuracy within tens of iterations, while the su-algorithm exhibits oscillatory behavior. On the ε -SVR problem, both algorithms outperform the PSG benchmark, with the r-algorithm attaining the smallest final objective value. Furthermore, the memory-saving initialization of the transformation matrix with a reduced matrix offers a practical strategy for large-scale settings.

Outline. Section 2 introduces the H-form r-algorithm and explains the motivation for developing the B-form r-algorithm. Section 3 describes the B-form r-algorithm in detail, including its methodology and the mechanism of space dilation. Section 4 presents the su-algorithm and highlights its key difference from the r-algorithm. Section 5 describes the numerical experiments conducted in this project using these two algorithms. Section 6 discusses the memory-saving initialization of the transform matrix. Section 7 reports and discusses the experimental results. Finally, Section 8 concludes the report.

2 H-form r-Algorithm

Let us consider the problem of minimizing a convex function $f(x)$, where $x \in \mathbb{R}^n$ is a vector of feasible variables. We denote the optimal value by $f^* = f(x^*)$, where x^* is an optimal solution. The r-algorithm is an iterative method that combines subgradient descent with space transformation. At each iteration, both the solution x_k and the transformation matrix H_k are updated, where $k = 0, 1, 2, \dots$.

The r-algorithm can be expressed in the H -form, as introduced in [9]. In this formulation, the method is written in terms of a symmetric definite matrix $H_k = B_k B_k^\top$, which resembles variable metric methods. The algorithm is then associated with the iterative generation of sequences $\{x_k\}_{k=0}^\infty$ and symmetric matrices $\{H_k\}_{k=0}^\infty$, according to the following rule:

$$x_{k+1} = x_k - h_k \frac{H_k g_f(x_k)}{\sqrt{(g_f(x_k))^\top H_k g_f(x_k)}}, \quad (1)$$

$$H_{k+1} = H_k + (\beta_k^2 - 1) \frac{H_k r_k r_k^\top H_k}{r_k^\top H_k r_k}, \quad k = 0, 1, \dots, \quad (2)$$

where

$$h_k \geq h_k^* = \arg \min_{h \geq 0} f \left(x_k - h \frac{H_k g_f(x_k)}{\sqrt{(g_f(x_k))^\top H_k g_f(x_k)}} \right), \quad (3)$$

$$\beta_k = \frac{1}{\alpha_k} < 1, \quad r_k = g_f(x_{k+1}) - g_f(x_k).$$

Although the r-algorithm can be expressed in the H -form, this formulation may suffer from numerical instability. In particular, the update of H_k tends to accumulate errors over iterations, and it is difficult to guarantee that H_k remains symmetric positive definite (SPD). To address these issues, the B -form r-algorithm is introduced, providing a more stable representation of the transformation.

3 B-form r-Algorithm

Let us consider the problem of minimizing a convex function $f(x)$, where $x \in \mathbb{R}^n$ is a vector of feasible variables. We denote the optimal value by $f^* = f(x^*)$, where x^* is an optimal solution. The r-algorithm is an iterative method that combines subgradient descent with space transformation. At each iteration, both the solution x_k and the transformation matrix B_k are updated, where $k = 0, 1, 2, \dots$.

At the initial iteration, the transformation matrix is given by $B_0 = I_n$. Therefore, the first update direction reduces to the normalized subgradient direction, and the step from x_0 to x_1 corresponds to a standard steepest descent step. The space transformation only takes effect from subsequent iterations.

3.1 Space Dilation Operator

Let a vector $\eta \in \mathbb{R}^n$, $\|\eta\| = 1$, and a number $\alpha \geq 0$ be fixed. Any vector $x \in \mathbb{R}^n$ can be decomposed into a component along a unit direction η , plus an orthogonal part, see Figure 1 [8]:

$$x = \gamma_\eta(x)\eta + d_\eta(x), \quad (4)$$

where

$$(\eta, d_\eta(x)) = 0 \quad (5)$$

From 4 and 5 we obtain

$$\gamma_\eta(x) = (x, \eta), \quad d_\eta(x) = x - (x, \eta)\eta.$$

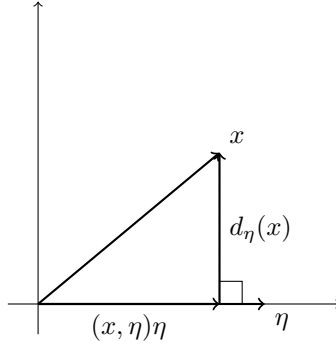


Figure 1: Decomposition of vector x into components parallel and orthogonal to η .

Definition 3.1 An operator $R_\beta(\eta)$ which transforms a vector x of the form 4 into

$$R_\beta(\eta)x = \beta\gamma_\eta(x)\eta + d_\eta(x)$$

is called an operator of space dilation along direction η with coefficient β . [8]

Substituting $\gamma_\eta(x)$ and $d_\eta(x)$ into $R_\beta(\eta)x$, we obtain

$$R_\beta(\eta)x = x + (\beta - 1)\eta(\eta, x)$$

Simplified,

$$R_\beta(\eta)x = x + (\beta - 1)(\eta\eta^\top)x = [I + (\beta - 1)\eta\eta^\top]x$$

Hence, the space dilation operator is [9]

$$R_\beta(\eta) = I + (\beta - 1)\eta\eta^\top. \quad (6)$$

The space dilation operator provides a way to modify the geometry of the space, which is the main idea behind the r-algorithm.

3.2 Update x_k

Given the current point x_k we compute a subgradient $g_f(x_k)$ and define the normalized direction in the transformed space as

$$\xi_k = \frac{B_k^\top g_f(x_k)}{\|B_k^\top g_f(x_k)\|}. \quad (7)$$

The descent direction is then given by

$$d_k = B_k \xi_k,$$

and the next iterate is

$$x_{k+1} = x_k - h_k B_k \xi_k, \quad (8)$$

where h_k is chosen using an adaptive step adjustment strategy in the line search.

3.3 Line Search: Adaptive step adjustment strategy

The step size h_k is selected using an adaptive step adjustment strategy along the direction $d_k = B_k \xi_k$. Instead of computing the exact line search to find the optimal step size h_k^*

$$h_k^* = \arg \min_{h \geq 0} f(x_k - h B_k \xi_k),$$

which can be computationally expensive, the r-algorithm employs an adaptive strategy to obtain a sufficiently good step size.

$$h_k \geq h_k^* = \arg \min_{h \geq 0} f(x_k - h B_k \xi_k), \quad (9)$$

The procedure is controlled by four parameters:

- $h_0 > 0$: initial step size,
- $q_1 \leq 1$: step reduction coefficient,
- $q_2 \geq 1$: step increase coefficient,
- n_h : frequency of step size increase.

Starting from the current step size h_k , the algorithm performs a one-dimensional search along the descent direction. If the descent condition is not satisfied, the step size is adjusted iteratively. If the step is too large, meaning the line search is finished in one step and does not produce sufficient descent, it is reduced by a coefficient q_1 . After n_h successful steps, the step size is increased by a coefficient q_2 to accelerate convergence.

The line search terminates when the descent direction is no longer effective. Specifically, the stopping condition is given by

$$(B_k^\top g_f(x_k))^\top (B_k^\top g_f(x_{k+1})) \leq 0.$$

This condition indicates that the angle between two successive subgradients in the transformed space is no longer acute, and therefore further movement along the current direction will not produce sufficient descent. Equivalently, this condition can be interpreted as checking whether the search direction ceases to be a descent direction at the new point.

3.4 Update B_k

After updating the x_k , the transformation matrix B_k is modified using a space dilation operator constructed from the difference of successive subgradients, see Figure 2:

$$r_k = g_f(x_{k+1}) - g_f(x_k). \quad (10)$$

The space dilation coefficient, which controls the strength of the space deformation, is defined as

$$\beta_k = \frac{1}{\alpha_k} \leq 1 \quad (11)$$

where $\alpha_k > 1$

The normalized dilation direction, which determines the direction along which the space is deformed, is defined as

$$\eta_k = \frac{B_k^\top r_k}{\|B_k^\top r_k\|} \quad (12)$$

The matrix B_k update is given by

$$B_{k+1} = B_k R_{\beta_k}(\eta_k), \quad (13)$$

This update modifies the geometry of the space so that the angle between successive subgradients is adjusted to improve descent efficiency, reducing the zigzagging behavior typically observed in subgradient methods and enhancing convergence.

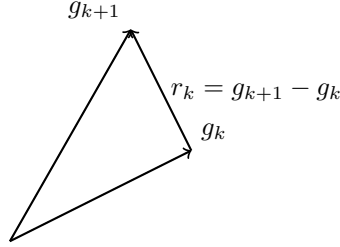
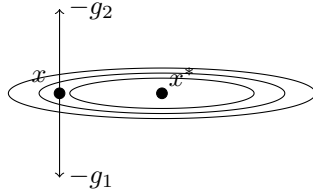


Figure 2: Difference of successive subgradients defining the dilation direction r_k .

(a) Ill-conditioned space



(b) After space dilation

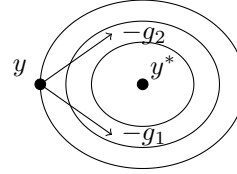


Figure 3: Space dilation with respect to the difference of two subgradients, where antsubgradients become the directions of function decrease

3.5 Stopping Criteria

The iterative process of the algorithm continues until one of several stopping criteria is satisfied. These criteria are essential to ensure both numerical stability and computational efficiency. There are two main parameters, ε_x and ε_g , that play a key role in this stopping criterion.

The algorithm terminates if the change between two successive iterations becomes sufficiently small.

$$\|x_{k+1} - x_k\| \leq \varepsilon_x$$

The algorithm also stops if the norm of the subgradient at the current point is small.

$$\|g_f(x_{k+1})\| \leq \varepsilon_g$$

A standard stopping rule is applied if the number of iterations exceeds a prescribed maximum $k_{\max}$, preventing excessive computation in difficult cases.

Below is the complete Definition of the B-form Algorithm:

Definition 3.2 *r*-algorithm for minimization of $f(x)$ is an iterative procedure of finding the sequence of n -dimensional vectors $\{x_k\}_{k=0}^\infty$ and the sequence of $(n \times n)$ -matrices $\{B_k\}_{k=0}^\infty$ by the following rule [9]:

$$\begin{aligned} x_{k+1} &= x_k - h_k B_k \xi_k, & B_{k+1} &= B_k R_{\beta_k}(\eta_k), & k &= 0, 1, 2, \dots, \\ \xi_k &= \frac{B_k^\top g_f(x_k)}{\|B_k^\top g_f(x_k)\|}, & h_k &\geq h_k^* = \arg \min_{h \geq 0} f(x_k - h B_k \xi_k), \\ \beta_k &= \frac{1}{\alpha_k} < 1, & \eta_k &= \frac{B_k^\top r_k}{\|B_k^\top r_k\|}, & r_k &= g_f(x_{k+1}) - g_f(x_k). \end{aligned}$$

This interpretation establishes a connection between the *r*-algorithm and classical variable metric methods. Motivated by this connection, the *su*-algorithm adopts a more direct approach by updating the transformation matrix using subgradient information, rather than through geometric space dilation. In contrast to the *r*-algorithm, which updates the transformation matrix through space dilation, the *su*-algorithm adopts a different strategy based on subgradient information.

4 su-Algorithm

In contrast to the *r*-algorithm, which updates the transformation matrix through space dilation, the *su*-algorithm adopts a different strategy based on subgradient information. Instead of modifying the geometry via a space dilation operator, the transformation matrix is updated directly using gradient-based information. Similar to the *r*-algorithm, the *su*-algorithm is an iterative method that updates both the solution x_k and the transformation matrix B_k at each iteration.

4.1 Update x_k

Given the current point x_k , we compute a subgradient $g_f(x_k)$ and define the normalized direction in the transformed space as

$$\xi_k = \frac{B_k^\top g_f(x_k)}{\|B_k^\top g_f(x_k)\|}.$$

The next iterate is

$$x_{k+1} = x_k - h_k B_k \xi_k,$$

where h_k is chosen using the same adaptive step adjustment strategy as in the *r*-algorithm. Moreover, the stopping criteria are the same.

4.2 Update B_k

The update of the transformation matrix is motivated by the variable-metric method proposed in [12]. In the original formulation, the matrix update is derived by applying a subgradient step to the function

$$\varphi_k(B_k) = f(x_k - h_k B_k B_k^\top g_f(x_k)).$$

Lemma 4.1 *Let the function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be convex on $\mathbb{R}^n$, and let the set $\mathcal{B} \subset \mathbb{R}^{n \times n}$ be convex. Then, the function $\varphi(B_k) = f(x - h_k B_k B_k^\top g_f(x_k))$ is weakly convex on $\mathcal{B}$, and [12]*

$$\partial \varphi(B_k) = \left\{ -h_k (\eta_k g_f(x_k)^\top + g_f(x_k) \eta_k^\top) B_k : \eta_k \in \partial f(x_k - h_k B_k B_k^\top g_f(x_k)) \right\}.$$

According to Lemma 5.1, applying a subgradient descent step to B_k gives the update rule

$$B_{k+1} = B_k - \theta_k \partial \varphi_k(B_k).$$

Hence, we use

$$B_{k+1} = B_k + \theta_k (\eta_k g_f(x_k)^\top + g_f(x_k) \eta_k^\top) B_k, \quad (14)$$

where $\theta_k > 0$ is the effective matrix-update step-size parameter. Here, the factor h_k is absorbed into θ_k .

In our implementation, the iterate update x_{k+1} uses the normalized search direction

$$\xi_k = \frac{B_k^\top g_f(x_k)}{\|B_k^\top g_f(x_k)\|}.$$

Thus, the practical update for x_k is slightly different from the original theoretical form used to motivate the matrix-update rule. The vector η_k is chosen as a normalized subgradient at the new point:

$$\eta_k = \frac{g_f(x_{k+1})}{\|g_f(x_{k+1})\|}.$$

This update incorporates first-order information from successive subgradients and allows the transformation matrix to adapt to the local structure of the objective function.

Below is the complete form of the su-algorithm:

Definition 4.1 *The su-algorithm for minimization of $f(x)$ is an iterative procedure that generates the sequences $\{x_k\}_{k=0}^\infty$ and $\{B_k\}_{k=0}^\infty$ according to*

$$\begin{aligned} x_{k+1} &= x_k - h_k B_k \xi_k, \\ B_{k+1} &= B_k + \theta_k (\eta_k g_f(x_k)^\top + g_f(x_k) \eta_k^\top) B_k, \end{aligned}$$

where

$$\xi_k = \frac{B_k^\top g_f(x_k)}{\|B_k^\top g_f(x_k)\|}, \quad \eta_k = \frac{g_f(x_{k+1})}{\|g_f(x_{k+1})\|}.$$

5 Experiments

We evaluate the performance of the two algorithms on a variety of optimization problems and compare their effectiveness. In addition, for selected problems, the solution from Portfolio Safeguard (PSG) is used as a benchmark. To test the performance of these two algorithms, we design several test problems, including smooth ill-conditioned functions, nonsmooth ill-conditioned functions, machine learning problems such as ε -SVR, regression error minimization problem, as well as the CVaR portfolio optimization problem.

Following the recommendations provided in the paper [9], the algorithmic parameters are chosen to ensure stable and efficient performance across different problem settings. In particular, for the r-algorithm with adaptive step adjustment applied to nonsmooth problems, the space dilation coefficient is typically selected as $\alpha \in [2, 4]$, with an initial step size $h_0 = 1.0$. The step adjustment parameters are chosen as $q_1 = 1.0$ and $q_2 \in [1.1, 1.2]$, with the step increase frequency parameter $n_h \in [2, 3]$. When minimizing smooth functions, similar parameter values are used, except that the step reduction factor is taken as $q_1 \in [0.8, 0.95]$ to improve the accuracy of the line search. This additional refinement enhances convergence speed for smooth problems. The stopping tolerances are selected as $\varepsilon_x, \varepsilon_g \approx 10^{-6}$ to 10^{-5} , which are sufficient to obtain solutions close to the optimum even for ill-conditioned problems. Under these parameter settings, the algorithms typically achieve high accuracy with a limited number of descent steps. However, in our experiments, we can make small changes to these parameters in each experiment in order to calculate a better solution.

By default, we set the parameters as follows: $h_0 = 1.0$, $q_1 = 0.9$, $q_2 = 1.2$, $nh = 2.5$, $epsg = 10^{-8}$, $epsx = 10^{-8}$, $maxitn = 1000$, $intp = 50$. Here, `calcfg` denotes the function that returns the objective value f and gradient g for a given point x , while x_0 is the initial starting point. The parameter $\alpha > 0$ is the space dilation coefficient, h_0 is the initial step size, $q_1 \leq 1$ is the step-shrinking factor, and $q_2 \geq 1$ is the step-expansion factor. Further, `epsg` is the stopping tolerance for the gradient norm, `epsx` is the stopping tolerance for the step size, `maxitn` is the maximum number of iterations, and `intp` specifies that iteration information is printed every `intp` iterations.

5.1 Smooth ill-conditioned function

We consider the quadratic function

$$f_{quad}(x_1, x_2) = x_1^2 + 100x_2^2, \quad (15)$$

which is a classic example of an ill-conditioned problem due to the large difference in curvature along the two coordinate directions. Its gradient is given by

$$\mathbf{g}_{quad}(x_1, x_2) = \begin{pmatrix} 2x_1 \\ 200x_2 \end{pmatrix}. \quad (16)$$

For the numerical experiments, the initial point is set to $\mathbf{x}_0 = (1, 0.2)$. The parameter in the r-algorithm is chosen as $\alpha = 2.5$, while the subgradient algorithm uses $\theta = 0.001$.

5.2 Nonsmooth ill-conditioned function

We also consider the nonsmooth function

$$f_{abs}(x_1, x_2) = |x_1| + 10|x_2|, \quad (17)$$

which exhibits both nonsmoothness and anisotropic scaling, making it challenging for standard gradient-based methods. A subgradient of $f_{abs}(x_1, x_2)$ is given by

$$\mathbf{g}_{abs}(x_1, x_2) = \begin{cases} \begin{pmatrix} 1 \\ 10 \end{pmatrix}, & x_1 > 0, x_2 > 0, \\ \begin{pmatrix} 1 \\ -10 \end{pmatrix}, & x_1 > 0, x_2 < 0, \\ \begin{pmatrix} -1 \\ 10 \end{pmatrix}, & x_1 < 0, x_2 > 0, \\ \begin{pmatrix} -1 \\ -10 \end{pmatrix}, & x_1 < 0, x_2 < 0. \end{cases} \quad (18)$$

For the numerical experiments, the initial point is set to $\mathbf{x}_0 = (1, 0.2)$. The parameter in the r-algorithm is chosen as $\alpha = 2.5$, while the subgradient algorithm uses $\theta = 0.001$.

5.3 ε -Support Vector Regression (ε -SVR)

We also consider the ε -insensitive support vector regression problem

$$\min_{\mathbf{w}} \quad \mathbb{E}[(|\mathbf{z}(\mathbf{w})| - \varepsilon)_+] + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (19)$$

where

$$\mathbf{z}(\mathbf{w}) = y - \mathbf{x}^\top \mathbf{w}$$

is the residual.

In the finite-sample setting, this problem 19 becomes

$$\min_{\mathbf{w}} \quad \frac{1}{N} \sum_{i=1}^N \max(|z_i(\mathbf{w})| - \varepsilon, 0) + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (20)$$

where

$$z_i(\mathbf{w}) = y_i - \mathbf{x}_i^\top \mathbf{w}, \quad i = 1, \dots, N.$$

The first term is the ε -insensitive loss, which ignores errors within the margin ε , while the second term is an ℓ_2 -regularization term.

A subgradient of the objective function in problem (20) is given by

$$\mathbf{g}_\varepsilon(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i^\varepsilon(\mathbf{w}) + \lambda \mathbf{w} \quad (21)$$

, where

$$\mathbf{g}_i^\varepsilon(\mathbf{w}) = \begin{cases} \begin{cases} -\mathbf{x}_i, & \text{if } z_i(\mathbf{w}) > 0, \\ \mathbf{x}_i, & \text{if } z_i(\mathbf{w}) < 0, \end{cases} & \text{if } |z_i(\mathbf{w})| > \varepsilon, \\ \mathbf{0}, & \text{if } |z_i(\mathbf{w})| \leq \varepsilon. \end{cases} \quad (22)$$

We generate a simulated dataset with $N = 5000$ samples and $d = 10$ features. The covariates are independently drawn from the uniform distribution $X_{ij} \sim \text{Unif}(-1, 1)$. The true coefficient vector is set to $\mathbf{w}_{\text{true}} = (1, \dots, 1)^\top$, and the noise term is generated as $e_i \sim \text{Laplace}(0, 1)$. The response is then constructed according to

$$\mathbf{y} = \mathbf{X}\mathbf{w}_{\text{true}} + \mathbf{e}.$$

For the optimization setup, the initial point is chosen as $\mathbf{w}_0 = \mathbf{0}$, with $\varepsilon = 0.05$ and $\lambda = \frac{1}{2N}$. In addition, the parameter for the r-algorithm is set to $\alpha = 2.5$, while the parameter for the subgradient algorithm is taken as $\theta = 0.001$. We also solve this problem in PSG, the ε -insensitive regression loss is represented through the partial moment function `pm_pen`, while the ℓ_2 -regularization term is incorporated separately through `quadratic`. The PSG formulation is solved by the `BULDOZER` solver with precision level 9.

5.4 Regression Error Minimization Problem

We consider the regression error minimization problem

$$\min_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \min_{\mathbf{w}} \max \left(\mathbb{E}[\max(0, z(\mathbf{w}))], \mathbb{E}[\max(0, -z(\mathbf{w}))] \right), \quad (23)$$

where

$$z(\mathbf{w}) = y - \mathbf{x}^\top \mathbf{w}$$

is the residual.

In the finite-sample setting, this problem 23 becomes

$$\min_{\mathbf{w}} \mathcal{E}(\mathbf{w}) = \min_{\mathbf{w}} \max \left(\frac{1}{N} \sum_{i=1}^N \max(0, z_i(\mathbf{w})), \frac{1}{N} \sum_{i=1}^N \max(0, -z_i(\mathbf{w})) \right), \quad (24)$$

where

$$z_i(\mathbf{w}) = y_i - \mathbf{x}_i^\top \mathbf{w}, \quad i = 1, \dots, N.$$

Here, $X \in \mathbb{R}^{N \times d}$ denotes the design matrix, $\mathbf{x}_i^\top$ is its i -th row, and $\mathbf{y} \in \mathbb{R}^N$ is the response vector. Define

$$E_+(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \max(0, z_i(\mathbf{w})), \quad E_-(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \max(0, -z_i(\mathbf{w})). \quad (25)$$

A subgradient of the objective function in problem (24) is given by

$$\mathbf{g}_{\text{error}}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \mathbf{g}_i^{\text{error}}(\mathbf{w}). \quad (26)$$

, where

$$\mathbf{g}_i^{error}(\mathbf{w}) = \begin{cases} \begin{cases} -\mathbf{x}_i, & \text{if } z_i(\mathbf{w}) > 0, \\ \mathbf{0}, & \text{if } z_i(\mathbf{w}) \leq 0, \end{cases} & \text{if } E_+(\mathbf{w}) > E_-(\mathbf{w}), \\ \begin{cases} \mathbf{x}_i, & \text{if } z_i(\mathbf{w}) < 0, \\ \mathbf{0}, & \text{if } z_i(\mathbf{w}) \geq 0, \end{cases} & \text{if } E_+(\mathbf{w}) < E_-(\mathbf{w}), \\ \mathbf{0}, & \text{if } E_+(\mathbf{w}) = E_-(\mathbf{w}). \end{cases} \quad (27)$$

We generate a simulated dataset with $N = 5000$ samples and $d = 10$ features. The covariates are independently drawn from the uniform distribution $X_{ij} \sim \text{Unif}(-1, 1)$. The true coefficient vector is set to $\mathbf{w}_{\text{true}} = (1, \dots, 1)^\top$, and the noise term is generated as $e_i \sim \text{Laplace}(0, 1)$. The response is then constructed according to

$$\mathbf{y} = X\mathbf{w}_{\text{true}} + \mathbf{e}.$$

For the optimization setup, the initial point is chosen as $\mathbf{w}_0 = \mathbf{0}$. In addition, the parameter for the r-algorithm is set to $\alpha = 2.5$, while the parameter for the subgradient algorithm is taken as $\theta = 0.05$. We also solve this problem in PSG. In the problem statement, `super_err` is the objective variable representing the superexpectation error [3]. The PSG formulation is solved by the `BULDOZER` solver with precision level 9.

5.5 CVaR Portfolio Optimization Problem

We consider a portfolio optimization problem based on the Conditional Value-at-Risk (CVaR) criterion under a simplex constraint. Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ denote the matrix of asset returns, where each row $\mathbf{x}_i$ represents the return vector in scenario i , and let $\mathbf{w} \in \mathbb{R}^d$ be the portfolio weight vector. The portfolio loss in scenario i is defined as

$$L_i(\mathbf{w}) = -\mathbf{x}_i^\top \mathbf{w}, \quad i = 1, \dots, N. \quad (28)$$

Accordingly, the vector of sample losses is given by

$$\mathbf{L}(\mathbf{w}) = (L_1(\mathbf{w}), L_2(\mathbf{w}), \dots, L_N(\mathbf{w}))^\top.$$

The objective function combines the expected loss and the CVaR of the loss distribution. We minimize this function over the simplex

$$\mathcal{W} = \{\mathbf{w} \in \mathbb{R}^d : \mathbf{w} \geq 0, \mathbf{1}^\top \mathbf{w} = 1\}. \quad (29)$$

The optimization problem is formulated as

$$\min_{\mathbf{w} \in \mathcal{W}} \{\mathbb{E}[\mathbf{L}(\mathbf{w})] + \lambda \text{CVaR}_{\hat{\alpha}}(\mathbf{L}(\mathbf{w}))\}. \quad (30)$$

where $\lambda > 0$ is a risk-aversion parameter and $\hat{\alpha} \in (0, 1)$ is the confidence level.

The Value-at-Risk (VaR) is approximated by the $\hat{\alpha}$ -quantile of the sample losses, and the CVaR is computed as [6]

$$\text{CVaR}_{\hat{\alpha}}(\mathbf{L}(\mathbf{w})) = C + \frac{1}{1 - \hat{\alpha}} \frac{1}{N} \sum_{i=1}^N \max(L_i(\mathbf{w}) - C, 0),$$

where C denotes the VaR.

A subgradient of the objective function 30 is constructed from three parts. The contribution of the expected loss term is

$$\mathbf{g}_{EL}(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \mathbf{x}_i, \quad (31)$$

The contribution of the CVaR loss term is

$$\mathbf{g}_C(\mathbf{w}) = \frac{\lambda}{(1 - \hat{\alpha})N} \sum_{i=1}^N g_i^C(\mathbf{w}), \quad (32)$$

where

$$\mathbf{g}_i^C(\mathbf{w}) = \begin{cases} -\mathbf{x}_i, & \text{if } L_i(\mathbf{w}) > C, \\ \mathbf{0}, & \text{if } L_i(\mathbf{w}) \leq C. \end{cases} \quad (33)$$

To enforce the simplex constraint, $\mathbf{w} \geq 0$ and $\sum_{j=1}^d w_j = 1$, the algorithm adds a penalty-type projection term based on the distance between $\mathbf{w}$ and its projection onto the simplex [5]. Specifically, if $\Pi_{\mathcal{W}}(\mathbf{w})$ denotes the simplex projection, then the additional term is

$$M \frac{\mathbf{w} - \Pi_{\mathcal{W}}(\mathbf{w})}{\|\mathbf{w} - \Pi_{\mathcal{W}}(\mathbf{w})\|},$$

with the convention that this term is zero when $\mathbf{w}$ already lies on the simplex. Here $M > 0$ is a large penalty parameter. This yields a nonsmooth constrained optimization problem suitable for testing subgradient-type methods.

A subgradient of the objective function in problem 30 is given by

$$\mathbf{g}_{cvar}(\mathbf{w}) = \mathbf{g}_{EL}(\mathbf{w}) + \mathbf{g}_C(\mathbf{w}) + M \frac{\mathbf{w} - \Pi_{\mathcal{W}}(\mathbf{w})}{\|\mathbf{w} - \Pi_{\mathcal{W}}(\mathbf{w})\|} \quad (34)$$

We generate a simulated asset-return dataset with $N = 5000$ observations and $d = 10$ assets. The entries of the return matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ are independently drawn from the uniform distribution

$$X_{ij} \sim \text{Unif}(-1, 1), \quad i = 1, \dots, N, \quad j = 1, \dots, d.$$

Each row X_i is treated as the asset-return vector in scenario i .

For the optimization setup, the initial point is chosen as $\mathbf{w}_0 = [0.3, 1, 0.2, 0.1, 1, 0, 0, 0, 0, 0.2]$, with the confidence level $\hat{\alpha} = 0.95$. In addition, the parameter for the r-algorithm is set to $\alpha = 2.5$, while the parameter for the su-algorithm is taken as $\theta = 0.05$. The penalty parameter $M = 10$ for both algorithms. We also solve this problem in PSG. In the problem statement, the CVaR is represented through the function `cvar.risk`. The PSG formulation is solved by the BULDOZER solver with precision level 9.

6 Memory-Saving Initialization of the Matrix B_k

In the original implementation, the transformation matrix was initialized as the full identity matrix,

$$B_0 = I_n.$$

To reduce memory usage, we replace this initialization by a structured matrix of size $n \times m$, where $m \ll n$. Specifically, the first m rows form an identity block, while the remaining rows are assigned to a single pivot column. Thus, the initial matrix B_0 is given by

$$B_0 = \begin{pmatrix} I_m \\ \mathbf{e}_p^\top \\ \vdots \\ \mathbf{e}_p^\top \end{pmatrix} \in \mathbb{R}^{n \times m},$$

where I_m is the $m \times m$ identity matrix and $\mathbf{e}_p$ is the p -th coordinate vector. In our implementation, the pivot column is chosen as $p = m$ by default.

This modification replaces the storage of a full $n \times n$ matrix by a reduced $n \times m$ matrix, thereby saving memory when $m \ll n$. When $B_k \in \mathbb{R}^{n \times m}$, we have $B_k^\top g_f(x_k) \in \mathbb{R}^m$, so $\eta_k \in \mathbb{R}^m$ and $R_{\beta_k}(\eta_k) \in \mathbb{R}^{m \times m}$. Hence, the update $B_{k+1} = B_k R_{\beta_k}(\eta_k)$ remains dimensionally valid. The matrix B_k continues to serve as the transformation matrix in the B-form r-algorithm and su-algorithm, and the subsequent update formula remains unchanged.

7 Results

Let $f_k = f(x_k)$ denote the objective value at iteration k , where x_k is the current iterate. Let $f_k^r := \min_{0 \leq j \leq k} f(x_j)$ denote the best-so-far objective value up to iteration k . Here, f^* denotes the optimal objective value and x^* denotes the optimal solution of the problem. Let f^{r*} denote the corresponding reference optimal value for the best-so-far sequence.

7.1 Result for the Smooth ill-conditioned function

For the quadratic test problem (15), the numerical results are reported in Table 1.

f(x1, x2)=x1^2+100x2^2	Optimal result f(x*)
r - alg	4.08E-16
su - alg	9.16E-18
alpha=2.5, theta = 0.005, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxIt=1000	

Table 1: Optimal objective values obtained by the r-algorithm and su-algorithm for the quadratic problem.

Since the true minimizer of this function is $x_1 = 0$, $x_2 = 0$ with optimal value $f(x^*) = 0$, these results show that both methods are highly accurate and successfully identify the optimal solution. The su-algorithm produced a solution slightly closer to the true minimizer and achieved a smaller final objective value than the r-algorithm under the same parameter setting.

Table 2 compares the iteration performance of the r-algorithm and the su-algorithm in terms of the objective gap $f_k - f^*$ and the best-so-far objective gap $f_k^r - f^{r*}$, respectively.

Quadratic			Quadratic		
Gap	# of iterations		Gap	# of iterations	
	r-alg	su-alg		r-alg	su-alg
10^{-1}	10	25	10^{-1}	9	24
10^{-2}	10	35	10^{-2}	9	34
10^{-3}	18	45	10^{-3}	17	44
10^{-4}	21	55	10^{-4}	20	54
10^{-5}	24	63	10^{-5}	23	62
10^{-6}	26	73	10^{-6}	25	72
10^{-7}	27	81	10^{-7}	26	80
10^{-8}	32	87	10^{-8}	31	86

objective gap

best-so-far objective gap

Table 2: Number of iterations required by the r-algorithm and su-algorithm to attain different optimality gap levels for the smooth ill-conditioned quadratic problem. The left table reports the objective gap, while the right table reports the best-so-far objective gap.

In Figure 4, the r-algorithm reduces the objective gap substantially faster than the su-algorithm. Although the convergence path of the r-algorithm is somewhat nonmonotone in the early iterations, its objective gap decreases rapidly and reaches 10^{-8} in 32 iterations. By contrast, the su-algorithm shows a smoother but much slower decrease, requiring 87 iterations to attain the same gap level.

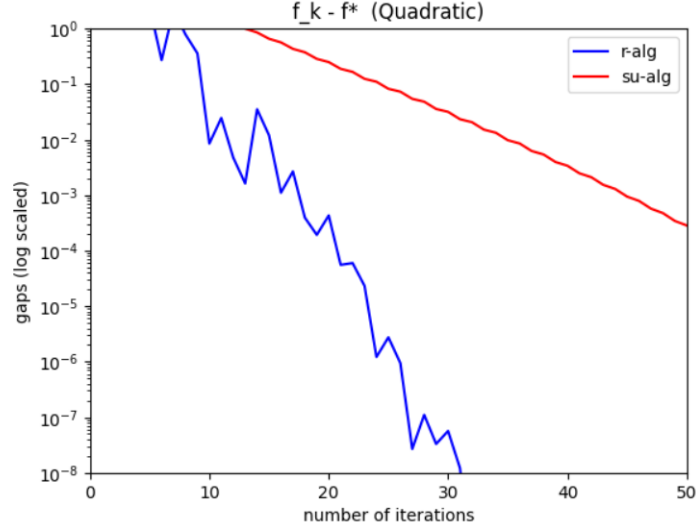


Figure 4: Objective gap $f_k - f^*$ for the quadratic problem.

Figure 5 shows the evolution of the best objective value found so far. Again, the r-algorithm achieves a substantially faster reduction in the gap. The su-algorithm also improves over time, but at a much slower rate. Therefore, the figures and tables consistently indicate that the r-algorithm outperforms the su-algorithm on this quadratic example in terms of convergence speed and numerical efficiency.

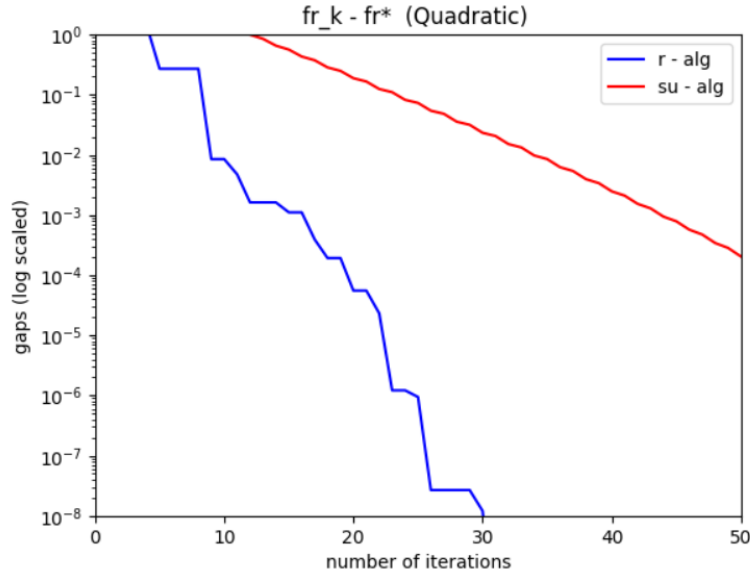


Figure 5: Best-so-far objective gap $f_k^r - f^{r*}$ for the quadratic problem.

7.2 Result for the Nonsmooth ill-conditioned function

For the nonsmooth ill-conditioned problem (17), the numerical results are reported in Table 3.

$f(x_1, x_2)= x_1 +10 x_2 $	Optimal result $f(x^*)$
r - alg	1.52E-08
su - alg	1.11E-10
alpha=2.5, theta = 0.001, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitn=1000	

Table 3: Optimal objective values obtained by the r-algorithm and su-algorithm for the nonsmooth ill-conditioned problem.

Since the true minimizer of this function is $x_1 = 0, x_2 = 0$ with optimal value $f(x^*) = 0$, these results show that both methods are able to identify the optimal solution with high accuracy. The su-algorithm produced a solution slightly closer to the true minimizer and achieved a smaller final objective value than the r-algorithm under the same parameter setting.

Table 4 compares the iteration performance of the r-algorithm and the su-algorithm in terms of the objective gap $f_k - f^*$ and the best-so-far objective gap $f_k^r - f^{r*}$, respectively.

Absolute			Absolute		
Gap	# of iterations		Gap	# of iterations	
	r-alg	su-alg		r-alg	su-alg
10^{-1}	14	16	10^{-1}	13	15
10^{-2}	23	30	10^{-2}	22	29
10^{-3}	30	58	10^{-3}	29	57
10^{-4}	43	96	10^{-4}	42	95
10^{-5}	48	96	10^{-5}	47	95
10^{-6}	52	408	10^{-6}	51	407
10^{-7}	58	446	10^{-7}	57	445
10^{-8}	63	446	10^{-8}	62	445

objective gap best-so-far objective gap

Table 4: Number of iterations required by the r-algorithm and su-algorithm to attain different optimality gap levels for the nonsmooth ill-conditioned problem. The left table reports the objective gap, while the right table reports the best-so-far objective gap.

In Figure 6, the r-algorithm reduces the objective gap substantially faster than the su-algorithm. Although the convergence path of the r-algorithm is slightly nonmonotone in the early iterations, its objective gap decreases rapidly overall and reaches the 10^{-7} level in 58 iterations, as reported in Table 4. By contrast, the su-algorithm exhibits highly oscillatory behavior and makes much slower progress. It requires 446 iterations to reach the same 10^{-7} gap level, indicating significantly slower and less stable convergence than the r-algorithm.

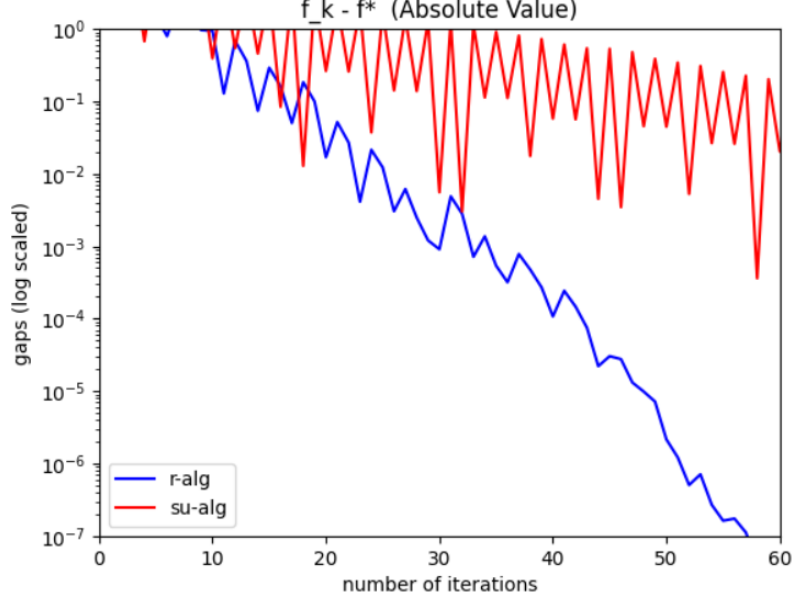


Figure 6: Objective gap $f_k - f^*$ for the nonsmooth problem.

In Figure 7, the r-algorithm reduces the best-so-far objective gap substantially faster than the su-algorithm. The r-algorithm achieves a much more pronounced decrease and reaches the 10^{-7} level in 57 iterations, as reported in Table 4. By contrast, the su-algorithm improves much more slowly and exhibits long stagnation phases, remaining nearly flat over a large number of iterations. In particular, it requires 445 iterations to attain the same 10^{-7} gap level, indicating significantly slower convergence than the r-algorithm.

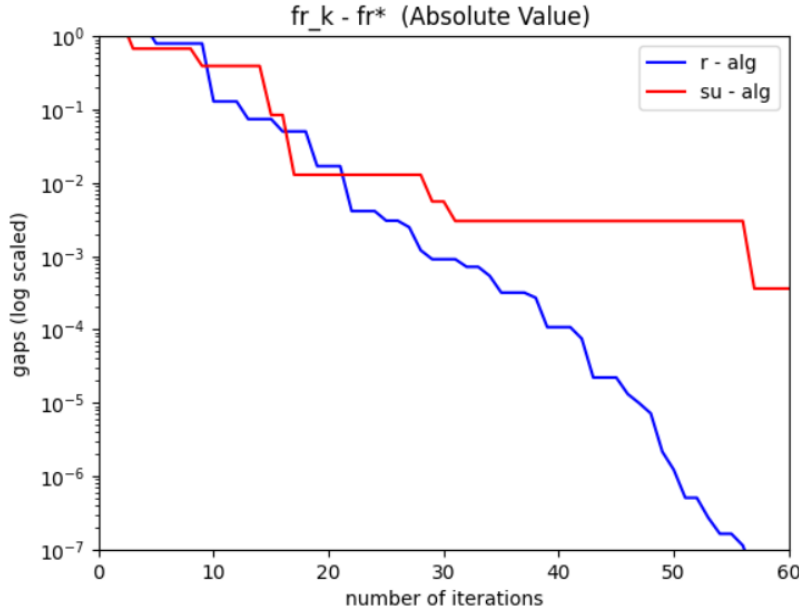


Figure 7: Best-so-far objective gap $f_k^r - f^{r*}$ for the nonsmooth problem.

7.3 Result for the ε -SVR problem

For the ε -SVR problem (20), the numerical results are reported in Table 5.

Epsilon -SVR	Optimal result $f(x^*)$
r - alg	0.9256773639
su - alg	0.9256773698
epsilon = 0.05, alpha=2.5, theta = 0.001, h0= 1.0, q1= 0.9, q2 = 1.2, nh =2.5, epsg=1e-8, epsx=1e-8, maxitn=1000	

Table 5: Optimal objective values obtained by the r-algorithm and su-algorithm for the ε -SVR problem. For comparison, the optimal objective value obtained by PSG is 0.92617938499.

The numerical results show that the r-algorithm and the su-algorithm produce very similar optimal solutions and objective values. In particular, both the r-algorithm and the su-algorithm attain the same final objective value, $f(x^*) = 0.9256773698$, up to 10 decimal places. For comparison, the PSG solver returns the best objective value, $f(x^*) = 0.92617938499$.

Figure 8 shows the computational protocol of the numerical experiments. The protocol consists of the raw execution output of the code, showing the iteration progress of the r-algorithm and the su-algorithm, together with the optimal solution and objective value.

```

***** Epsilon -SVR *****
===== r - alg L2 Penalty =====
itn   0   f   1.759608e+00 fr   1.759608e+00 nfg   1
itn  30   f   9.257144e-01 fr   9.256971e-01 nfg  47 lsa  46 lsm  4
itn  60   f   9.256774e-01 fr   9.256774e-01 nfg  82 lsa  35 lsm  2
itn  90   f   9.256774e-01 fr   9.256774e-01 nfg 131 lsa  49 lsm  5
itn 120   f   9.256774e-01 fr   9.256774e-01 nfg 188 lsa  57 lsm  4
Optimal solution: [0.01092073 0.97047724 0.98735716 0.99100563 1.00063979 1.04336375
1.03077055 1.00682996 1.01003232 0.96627639 1.01107957]
Final f(x): 0.9256773698720009
===== su - alg L2 Penalty =====
itn   0   f   1.759608e+00 fr   1.759608e+00 nfg   1
itn  30   f   9.257524e-01 fr   9.257028e-01 nfg  34 lsa  33 lsm  4
itn  60   f   9.256789e-01 fr   9.256784e-01 nfg  64 lsa  30 lsm  1
itn  90   f   9.256774e-01 fr   9.256774e-01 nfg  94 lsa  30 lsm  1
itn 120   f   9.256774e-01 fr   9.256774e-01 nfg 125 lsa  31 lsm  2
itn 150   f   9.256774e-01 fr   9.256774e-01 nfg 157 lsa  32 lsm  2
itn 180   f   9.256774e-01 fr   9.256774e-01 nfg 187 lsa  30 lsm  1
Optimal solution: [0.01094894 0.97045443 0.98736203 0.990989 1.00070155 1.04335123
1.03075188 1.00681991 1.00999795 0.96626018 1.0110098 ]
Final f(x): 0.9256773766341012

```

Figure 8: Computational protocol for the ε -SVR problem with ℓ_2 -regularization, showing the iteration history and final output of the r-algorithm and the su-algorithm. Here, **itn** denotes the iteration number, **f** denotes the current objective value, **fr** denotes the best-so-far objective value, **nfg** denotes the number of function and gradient evaluations, **lsa** denotes the accumulated number of line-search steps, and **lsm** denotes the maximum number of line-search steps in the current iteration. The reported **Optimal solution** is the optimal solution found by the algorithm, and **Final $f(x)$** is the corresponding optimal objective value.

Table 6 compares the iteration performance of the r-algorithm and the su-algorithm in terms of the objective gap $f_k - f^*$ and the best-so-far objective gap $f_k^r - f^{r*}$, respectively.

Epsilon-SVR (esplion = 0.05)			Epsilon-SVR (esplion = 0.05)		
Gap	# of iterations		Gap	# of iterations	
	r-alg	su-alg		r-alg	su-alg
10^{-1}	3	3	10^{-1}	2	2
10^{-2}	9	7	10^{-2}	8	6
10^{-3}	28	11	10^{-3}	27	10
10^{-4}	39	29	10^{-4}	38	28
10^{-5}	49	43	10^{-5}	48	42
10^{-6}	55	62	10^{-6}	54	61
10^{-7}	74	85	10^{-7}	73	84
10^{-8}	94	117	10^{-8}	93	116

objective gap

best-so-far objective gap

Table 6: Number of iterations required by the r-algorithm and su-algorithm to attain different optimality gap levels for the ε -SVR problem. The left table reports the objective gap, while the right table reports the best-so-far objective gap.

In Figure 9, both the r-algorithm and the su-algorithm reduce the objective gap effectively, but their convergence behavior differs across stages of the iteration process. The su-algorithm exhibits faster initial decrease, whereas the r-algorithm ultimately achieves a steeper reduction and reaches 10^{-8} gap in 94 iterations.

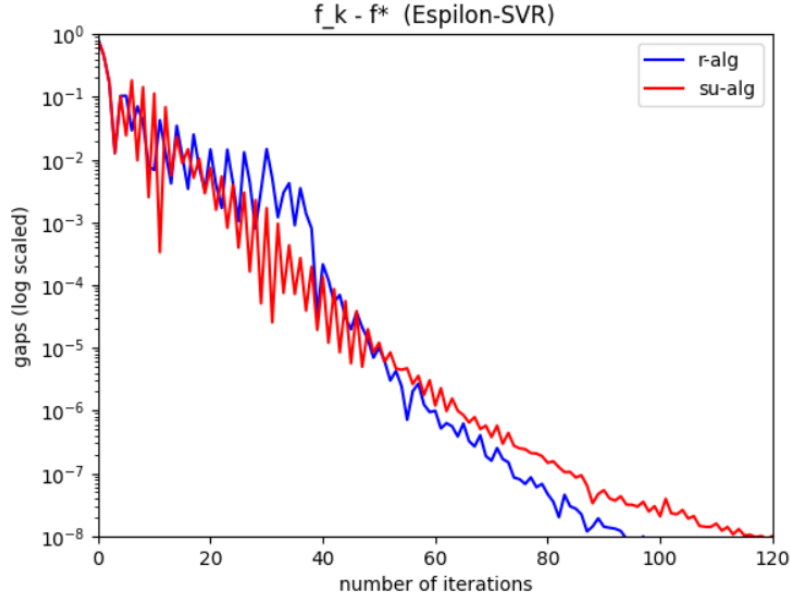


Figure 9: Objective gap $f_k - f^*$ for the ε -SVR problem.

Similar to the objective gap plot, both algorithms reduce the best-so-far objective gap effectively, but their convergence behavior differs across stages of the iteration process, as shown in Figure 10. The su-algorithm exhibits faster initial improvement, whereas the r-algorithm ultimately achieves a steeper reduction and reaches the 10^{-8} gap level in 93 iterations.

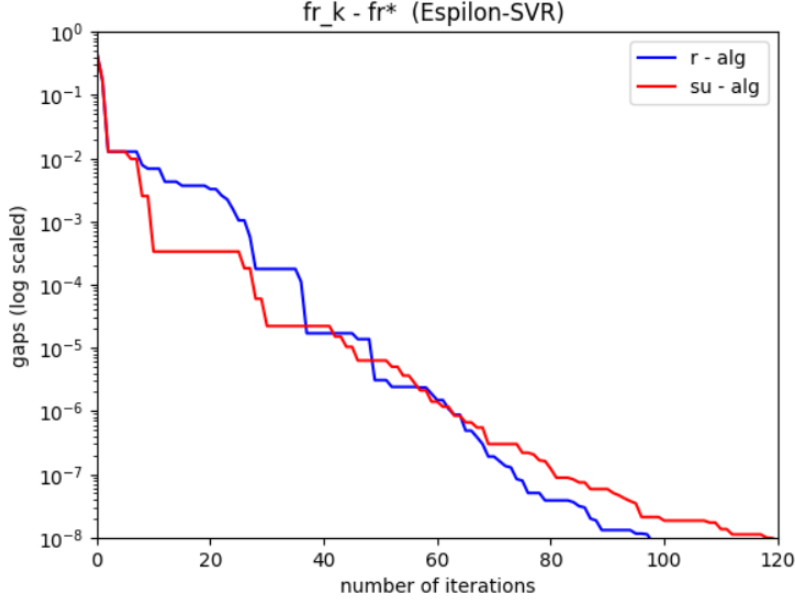


Figure 10: Best-so-far objective gap $f_k^r - f^{r*}$ for the ε -SVR problem.

7.4 Result for the Regression Error Minimization Problem

For the regression error minimization problem (23), the numerical results are reported in Table 7.

Regression Error	Optimal result $f(x^*)$
r - alg	0.4869638551
su - alg	0.4869638992
alpha=2.5, theta = 0.05, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitr=1000	

Table 7: Optimal objective values obtained by the r-algorithm and su-algorithm for the regression error minimization problem. For comparison, the optimal objective value obtained by PSG is 0.486963855073713.

The numerical results show that the r-algorithm and the su-algorithm produce very similar optimal solutions and objective values. In particular, both the r-algorithm and the su-algorithm attain the same objective value up to several decimal places. For comparison, the PSG solver returns the best objective value, $f(x^*) = 0.486963899208427$.

Table 8 compares the iteration performance of the r-algorithm and the su-algorithm in terms of the objective gap $f_k - f^*$ and the best-so-far objective gap $f_k^r - f^{r*}$, respectively.

Regression Error			Regression Error		
Gap	# of iterations		Gap	# of iterations	
	r-alg	su-alg		r-alg	su-alg
10^{-1}	7	8	10^{-1}	6	7
10^{-2}	11	16	10^{-2}	10	15
10^{-3}	29	20	10^{-3}	28	19
10^{-4}	44	76	10^{-4}	43	75
10^{-5}	55	92	10^{-5}	54	91
10^{-6}	64	108	10^{-6}	63	107
10^{-7}	99	139	10^{-7}	98	138
10^{-8}	136	NaN	10^{-8}	135	NaN

objective gap

best-so-far objective gap

Table 8: Number of iterations required by the r-algorithm and su-algorithm to attain different optimality gap levels for the regression error minimization problem. The left table reports the objective gap, while the right table reports the best-so-far objective gap.

In Figure 11, both the r-algorithm and the su-algorithm reduce the objective gap effectively, but their convergence behavior differs across stages of the iteration process. The su-algorithm exhibits slightly faster progress at some intermediate accuracy levels, reaching the 10^{-3} gap level in 20 iterations, compared with 29 iterations for the r-algorithm. However, the r-algorithm ultimately achieves a steeper reduction and reaches the 10^{-8} gap level in 136 iterations, whereas the su-algorithm does not attain the 10^{-8} level within the reported iteration range.

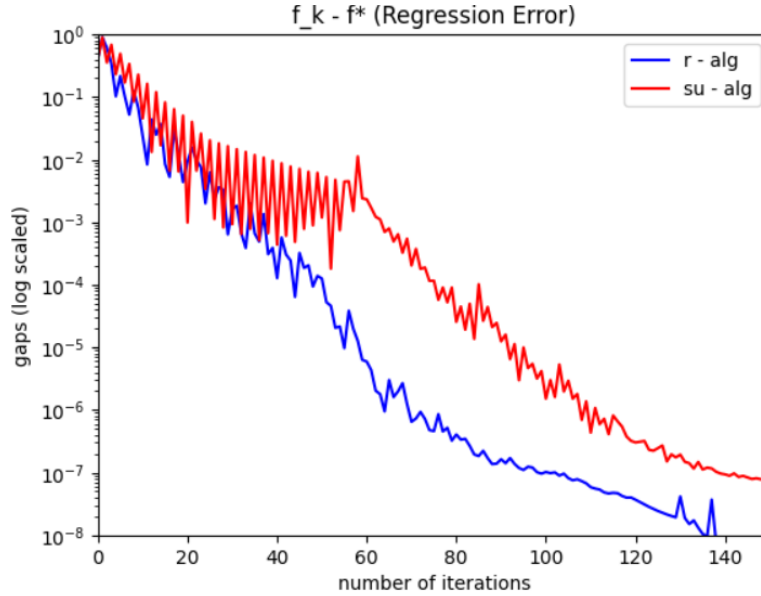


Figure 11: Objective gap $f_k - f^*$ for the Regression Error Minimization problem.

Similar to the objective gap plot, both algorithms reduce the best-so-far objective gap effectively, but their convergence behavior differs across stages of the iteration process, as shown in Figure 12. The su-algorithm again exhibits slightly faster initial improvement, reaching the 10^{-3} gap level in 19 iterations, compared with 28 iterations for the r-algorithm. However, the r-algorithm eventually shows a steeper reduction and reaches the 10^{-8} gap level in 135 iterations, while the su-algorithm does not attain the 10^{-8} level within the reported iteration range.

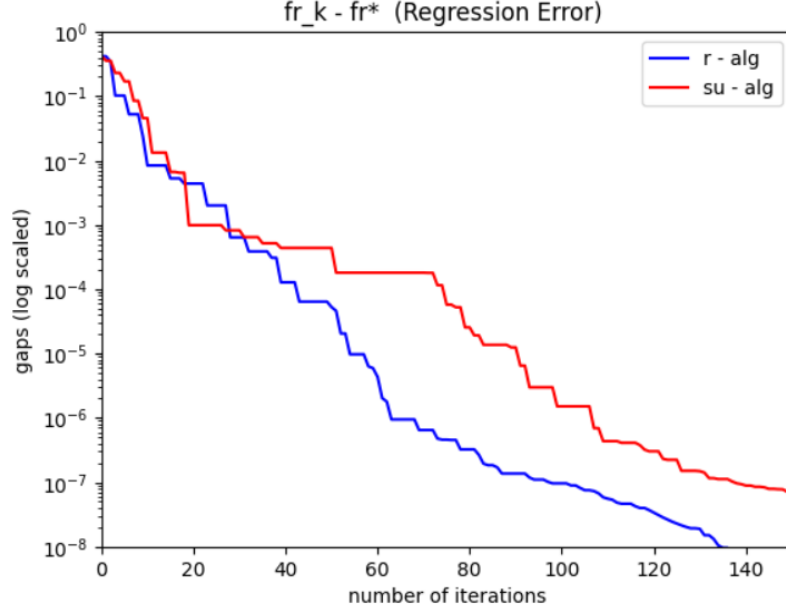


Figure 12: Best-so-far objective gap $f_k^r - f^{r*}$ for the Regression Error Minimization problem.

7.5 Result for the CVaR Portfolio Optimization Problem

For the CVaR portfolio optimization problem, the numerical results are reported in Table 9.

CVaR Port Opt	Optimal result $f(x^*)$
r - alg	0.374463252
su - alg	0.374238238

alpha=2.5, theta = 0.05, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitn=1000

Table 9: Optimal objective values obtained by the r-algorithm and su-algorithm for the CVaR portfolio optimization problem. For comparison, the optimal objective value obtained by PSG is 0.374463251947.

The numerical results show that the r-algorithm and the su-algorithm produce very similar optimal solutions and objective values. In particular, both methods attain objective values that are close to each other, with the r-algorithm giving $f(x^*) = 0.374463252$ and the su-algorithm giving $f(x^*) = 0.374238238$.

Table 10 compares the iteration performance of the r-algorithm and the su-algorithm in terms of the objective gap $f_k - f^*$.

CVaR Portfolio		
Gap	# of iterations	
	r-alg	su-alg
10^{-1}	17	12
10^{-2}	32	29
10^{-3}	39	32
10^{-4}	48	32
10^{-5}	61	92
10^{-6}	80	113
10^{-7}	117	141

objective gap

Table 10: Number of iterations required by the r-algorithm and su-algorithm to attain different optimality gap levels for the CVaR portfolio optimization problem.

In Figure 13, both the r-algorithm and the su-algorithm reduce the objective gap over the course of the iterations, but their convergence patterns are quite different. The r-algorithm exhibits a more stable and gradual decrease in the objective gap, with the curve trending downward in a relatively steady manner after the initial iterations. By contrast, the su-algorithm shows highly oscillatory behavior, with frequent sharp drops and rebounds throughout the iteration process.

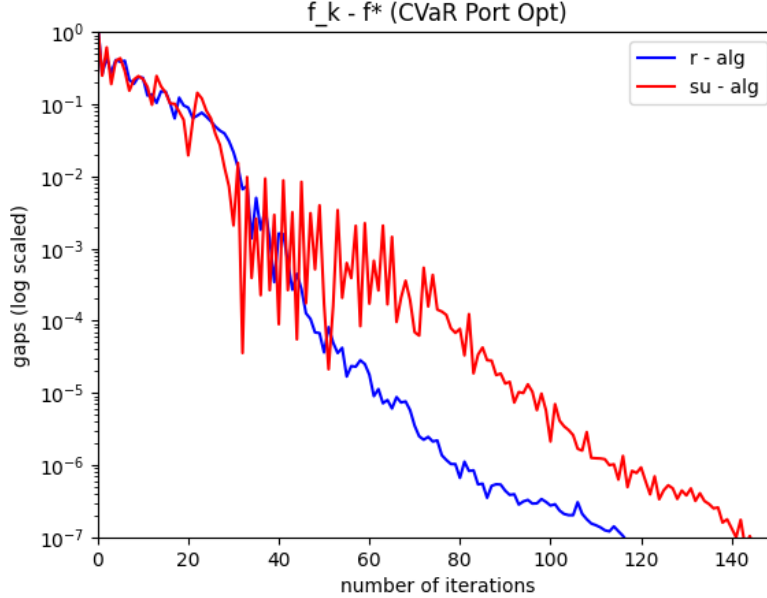


Figure 13: Objective gap $f_k - f^*$ for the CVaR portfolio optimization problem.

This behavior is mainly due to the sensitivity of the su-algorithm to the choice of the penalty parameter M . In addition, the step-size parameter θ has a substantial impact on its numerical performance. On the other hand, an inappropriate choice of these parameters may lead to poor performance and may even prevent the constraints from being properly enforced in the CVaR portfolio optimization problem 30; for example, the portfolio weights may fail to satisfy the budget constraint that their sum should equal 1. In contrast, the r-algorithm is much less sensitive to the choice of the penalty parameter M , whose effect on the numerical results is negligible. In practice, the parameters M and θ can be tuned to achieve better numerical performance.

7.6 Results for the Memory-Saving Initialization of the Matrix B_k

To evaluate the performance of the memory-saving initialization of the matrix B_k , we consider the ε -SVR problem in this experiment. The following tables report how the optimal solution and the corresponding objective value vary as the size parameter m increases, where m represents the reduced column dimension of the $n \times m$ matrix used to initialize B_k , up to the full dimension $d = 10$, for the r-algorithm and the su-algorithm, respectively.

From Figure 14, we can see that for both algorithms, the objective value decreases substantially when m increases from 1 to 2, and then remains nearly unchanged for larger values of m . Specifically, for the r-algorithm, the objective value drops from 0.07285 at $m = 1$ to approximately 0.050749 for $m \geq 2$. Moreover, we observe that the number of iterations is generally smaller for lower values of m . The su-algorithm exhibits the same trend, with the objective value decreasing from 0.06991 at $m = 1$ to approximately 0.0506 once $m \geq 2$. However, for the su-algorithm, the number of iterations does not exhibit a clear decreasing trend as m becomes smaller.

In addition, the corresponding optimal solutions for $m \geq 2$ are very similar across different values of m for both methods. This suggests that the memory-saving strategy has little influence on the final optimization result. Therefore, for large-scale problems, the proposed memory-saving initialization offers a practical way to reduce memory cost while preserving essentially almost the same optimization accuracy.

Epsilon -SVR using r - Alg				Epsilon -SVR using su - Alg			
m	f(x*)	itn	nfg	m	f(x*)	itn	nfg
1	0.07285	25	49	1	0.06991	492	694
2	0.050749	46	81	2	0.0506	352	462
3	0.050705	64	104	3	0.050623	1000	1575
4	0.050634	83	135	4	0.050563	1000	1586
5	0.050535	91	136	5	0.050463	1000	1605
6	0.05052	106	161	6	0.050449	1000	1604
7	0.050485	100	133	7	0.050353	1000	1609
8	0.050416	116	165	8	0.050399	182	189
9	0.050402	142	216	9	0.050267	1000	1620
10	0.050327	165	238	10	0.050322	183	190
11	0.050101	159	241	11	0.050101	182	189

alpha=2.5, h0= 1.0, q1= 0.9, q2= 1.2, nh= 3, epsg=1e-8, epsx=1e-11, maxitn=1000

theta=0.001, h0= 1.0, q1= 0.9, q2= 1.2, nh= 3, epsg=1e-8, epsx=1e-11, maxitn=1000

Figure 14: Results for the memory-saving initialization of the matrix B_k in r-alg and su-alg

8 Conclusion

This project studied and compared two algorithms — the r-algorithm and the su-algorithm — from both theoretical and experimental perspectives, evaluating their performance across five optimization problems: a smooth ill-conditioned quadratic function, a nonsmooth ill-conditioned absolute-value function, the ε -SVR problem, a regression error minimization problem, and a CVaR portfolio optimization problem. A comparison with the PSG benchmark shows that both algorithms are competitive on the ε -SVR and regression error problems, producing solutions of comparable quality. On the CVaR problem, the r-algorithm matches the PSG solution closely, while the su-algorithm's result depends more heavily on parameter tuning. Overall, the r-algorithm performs better than the su-algorithm in the five problems we solved.

9 Appendix

This section presents the results of the designed test functions in Section 5.1 and Section 5.2 used to compare the r-algorithm, the su-algorithm, and the Adam optimizer. Moreover, the result of the

Beale Function is presented at the end.

9.1 Result for Smooth ill-conditioned function

$f(x_1, x_2)=x_1^2+100x_2^2$	Optimal result $f(x^*)$
r - alg	4.08E-16
su - alg	9.16E-18
Adam	1.88E-16

alpha=2.5, theta = 0.005, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitn=1000

Figure 15: Optimal objective values obtained by the r-algorithm, su-algorithm, and Adam optimizer for the quadratic problem.

Quadratic			
Gap	# of iterations		
	r-alg	su-alg	Adam
10^{-1}	10	25	15
10^{-2}	10	35	26
10^{-3}	18	45	57
10^{-4}	21	55	84
10^{-5}	24	63	102
10^{-6}	26	73	126
10^{-7}	27	81	149
10^{-8}	32	87	170

objective gap

Quadratic			
Gap	# of iterations		
	r-alg	su-alg	Adam
10^{-1}	9	24	15
10^{-2}	9	34	26
10^{-3}	17	44	57
10^{-4}	20	54	84
10^{-5}	23	62	102
10^{-6}	25	72	126
10^{-7}	26	80	149
10^{-8}	31	86	170

best-so-far objective gap

Figure 16: Number of iterations required by the r-algorithm, su-algorithm, and Adam optimizer to attain different optimality gap levels for the smooth ill-conditioned quadratic problem. The left table reports the objective gap, while the right table reports the best-so-far objective gap.

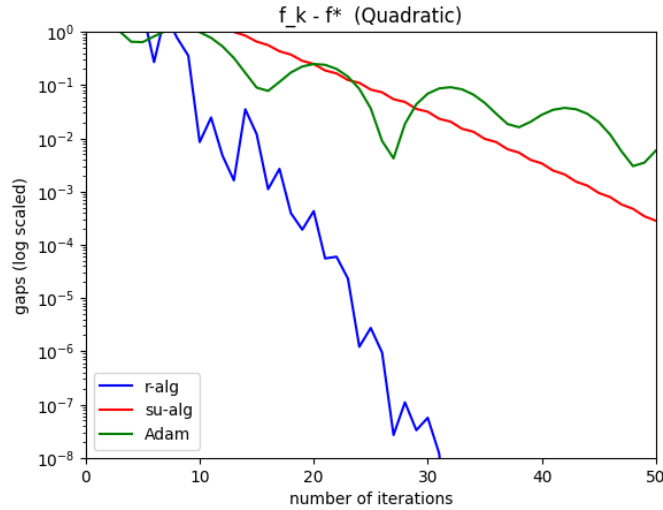


Figure 17: Objective gap $f_k - f^*$ for the quadratic problem.

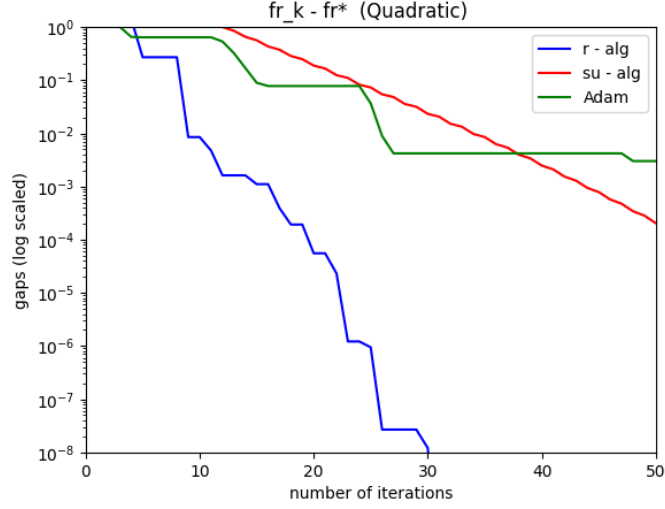


Figure 18: Objective gap $f_k^r - f^{r*}$ for the quadratic problem.

9.2 Result for the Nonsmooth ill-conditioned function

$f(x_1, x_2)= x_1 +10 x_2 $	Optimal result $f(x^*)$
r - alg	1.52E-08
su - alg	1.11E-10
Adam	3.63E-02

alpha=2.5, theta = 0.001, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitn=1000

Figure 19: Optimal objective values obtained by the r-algorithm, su-algorithm, and Adam optimizer for the nonsmooth ill-conditioned problem.

Absolute				Absolute			
Gap	# of iterations			Gap	# of iterations		
	r-alg	su-alg	Adam		r-alg	su-alg	Adam
10 ⁻¹	14	16	32	10 ⁻¹	13	15	32
10 ⁻²	23	30	NaN	10 ⁻²	22	29	NaN
10 ⁻³	30	58	NaN	10 ⁻³	29	57	NaN
10 ⁻⁴	43	96	NaN	10 ⁻⁴	42	95	NaN
10 ⁻⁵	48	96	NaN	10 ⁻⁵	47	95	NaN
10 ⁻⁶	52	408	NaN	10 ⁻⁶	51	407	NaN
10 ⁻⁷	58	446	NaN	10 ⁻⁷	57	445	NaN
10 ⁻⁸	63	446	NaN	10 ⁻⁸	62	445	NaN

objective gap

best-so-far objective gap

Figure 20: Optimal objective values obtained by the r-algorithm, su-algorithm, and Adam optimizer for the nonsmooth ill-conditioned problem.

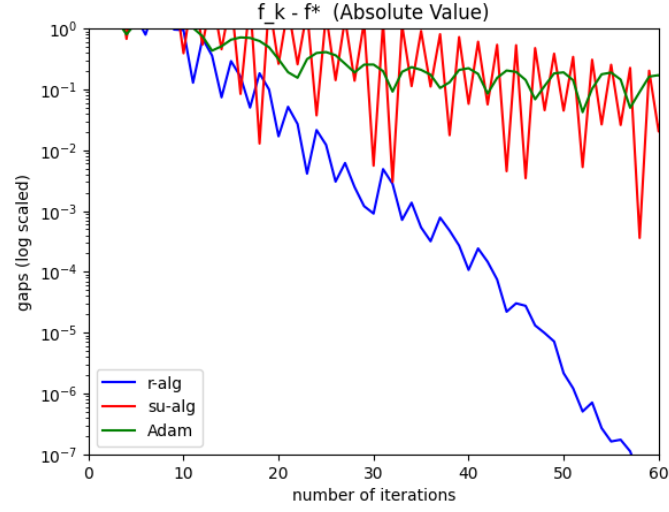


Figure 21: Objective gap $f_k - f^*$ for the nonsmooth ill-conditioned problem.

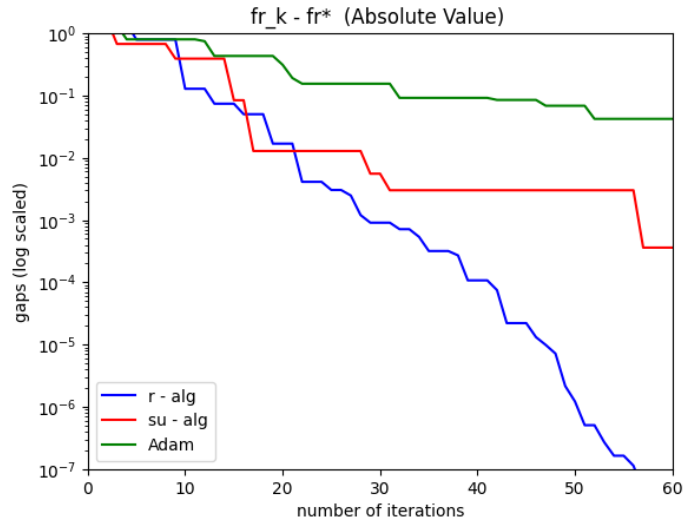


Figure 22: Objective gap $f_k^r - f^{r*}$ for the nonsmooth ill-conditioned problem.

9.3 Result for Beale Function

Beale Function	Optimal result $f(x^*)$
r - alg	4.99E-18
su - alg	2.14E-18
Adam	1.44E-20

alpha=2.5, theta = 0.005, h0= 1.0, q1= 0.9, q2 = 1.2, nh = 2.5, epsg=1e-8, epsx=1e-8, maxitn=1000

Figure 23: Optimal objective values obtained by the r-algorithm, su-algorithm, and Adam optimizer for the Beale Function.

Beale			
Gap	# of iterations		
	r-alg	su-alg	Adam
10^{-1}	4	17	43
10^{-2}	14	33	89
10^{-3}	15	56	165
10^{-4}	18	98	239
10^{-5}	21	130	305
10^{-6}	24	156	363
10^{-7}	30	182	416
10^{-8}	30	230	463

objective gap

Beale			
Gap	# of iterations		
	r-alg	su-alg	Adam
10^{-1}	3	16	43
10^{-2}	13	32	89
10^{-3}	14	55	165
10^{-4}	17	97	239
10^{-5}	20	129	305
10^{-6}	23	155	363
10^{-7}	29	181	416
10^{-8}	29	229	463

best-so-far objective gap

Figure 24: Optimal objective values obtained by the r-algorithm, su-algorithm, and Adam optimizer for the Beale Function.

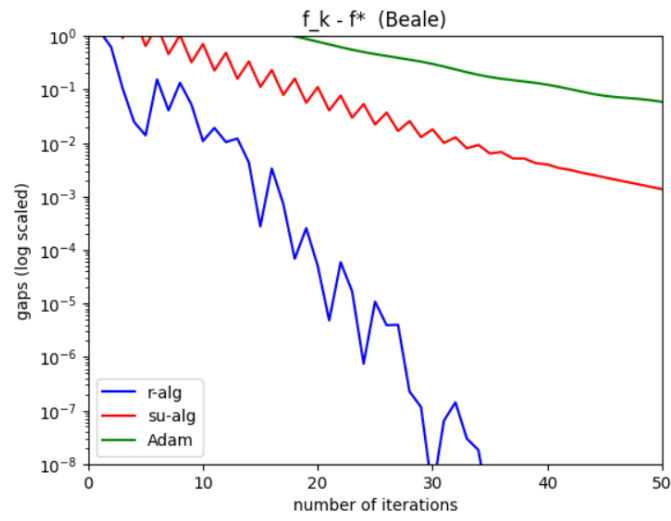


Figure 25: Objective gap $f_k - f^*$ for the Beale Function.

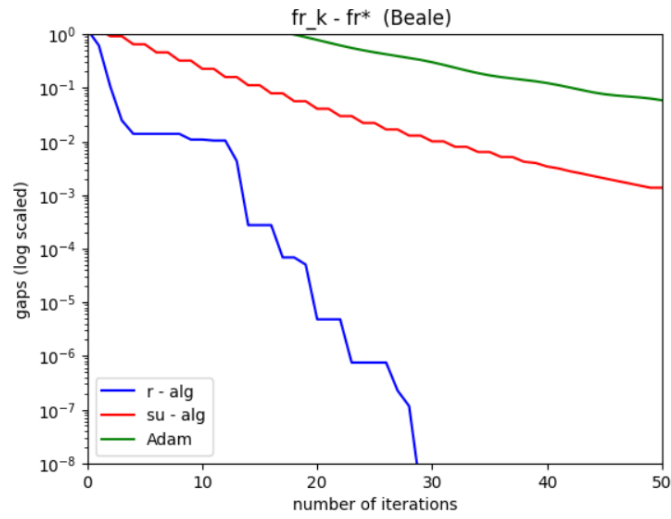


Figure 26: Objective gap $f_k^r - f^{r*}$ for the Beale Function.

References

- [1] American Optimal Decisions. AORDA: American Optimal Decisions. <https://aorda.com/>, 2019. Accessed: 2026-05-02.
- [2] A. Malandii and S. Uryasev. Support vector regression: Risk quadrangle framework. *arXiv preprint arXiv:2212.09178*, December 3 2024.
- [3] Anton Malandii and Stan Uryasev. Biased mean quadrangle and applications. *arXiv preprint arXiv:2603.26901*, March 2026. Submitted on 27 Mar 2026.
- [4] Yura Malitsky and Konstantin Mishchenko. Adaptive proximal gradient method for convex optimization. In *Advances in Neural Information Processing Systems*, 2024. 38th Conference on Neural Information Processing Systems.
- [5] Vladimir I. Norkin and Anton Y. Kozyriev. On shor’s r-algorithm for problems with constraints. *Cybernetics and Computer Technologies*, (3):16–22, 2023.
- [6] R. T. Rockafellar and S. Uryasev. Conditional Value-at-Risk for General Loss Distributions. *Journal of Banking and Finance*, 26:1443–1471, 2002.
- [7] R. Tyrrell Rockafellar and Stanislav Uryasev. Optimization of conditional value-at-risk. *The Journal of Risk*, 2(3):21–41, 2000.
- [8] Naum Z. Shor. *Nondifferentiable Optimization and Polynomial Problems*, volume 24 of *Non-convex Optimization and Its Applications*. Springer, Dordrecht, 1998.
- [9] Petro Stetsyuk. Theory and software implementations of shor’s r-algorithms*. *Cybernetics and Systems Analysis*, 53:1–12, 09 2017.
- [10] Sonja Surjanovic and Derek Bingham. Beale function. Virtual Library of Simulation Experiments: Test Functions and Datasets. Accessed: 2026-05-18.
- [11] Stanislav P. Uryasev. Adaptive variable metric algorithms for nonsmooth optimization problems. Technical Report WP-88-60, International Institute for Applied Systems Analysis (IIASA), Laxenburg, Austria, 1988.
- [12] Stanislav P. Uryasev. New variable-metric algorithms for nondifferentiable optimization problems. *Journal of Optimization Theory and Applications*, 71(2):311–333, 1991.